

Morphe

Plataforma de Processamento Digital de Sinais em FPGA
Manual do sistema: funcionamento e estrutura

Rascunho: capítulos 1 a 3 escritos; os demais, em estrutura

Antonio Nicassio Santos Lima
Estágio no LabPS/UEFS — supervisão: Prof. Armando S. Sanca

23 de setembro de 2026

Resumo

Conteúdo previsto. *Uma página: o que é o Morphe (operações de PDS executadas na FPGA de uma DE1-SoC e comparadas com a referência em software), a quem o manual se destina (professor, monitor e aluno que vai estender o sistema), o que foi herdado do TCC de Carlos Valadão e o que o estágio acrescentou. Os números que resumem a plataforma: cinco operações em hardware, sinais de qualquer tamanho por blocos, até ~ 92 dB de SNR na FFT, seis clientes simultâneos por placa sem perda de resposta.*

Sumário

1	Introdução	3
1.1	O que é o Morphe	3
1.2	Para que serve no ensino de PDS	3
1.3	Histórico	4
1.4	Visão geral	4
1.5	Como ler este manual	4
2	Arquitetura em três camadas	6
2.1	As camadas e o que cabe a cada uma	6
2.2	O caminho de uma operação	7
2.3	Onde está cada camada no repositório	9
3	O hardware	10
3.1	A placa DE1-SoC	10
3.2	Pontes e mapa de memória	11
3.3	Os aceleradores	12
3.3.1	O protocolo comum	12
3.3.2	Convolução e FIR: o <code>conv1d</code>	13
3.3.3	FFT e IFFT: o núcleo da Intel	13
3.3.4	IIR: a cascata de seções de segunda ordem	14
3.4	Periféricos ainda não usados: ADC e codec	15
4	O servidor na placa	17
5	O protocolo Morphe-TCP	18
6	Representação numérica	19
7	O aplicativo cliente	20
7.1	Janelas e módulos	20
7.2	Sinais maiores que o hardware: processamento em blocos	20
8	Várias placas, vários alunos	22
9	Preparação e operação do laboratório	23
10	Desenvolvimento e verificação	24
11	Limitações e trabalho futuro	25
A	Estrutura do repositório	26
B	Referência rápida	27

Lista de Figuras

1.1	Visão geral: os computadores do laboratório falam com as placas pela rede (TCP, porta 5000); a estação programa a FPGA pelo USB-Blaster e sobe o servidor por SSH.	5
2.1	As três camadas e os módulos de cada uma.	6
2.2	O caminho de uma operação, do clique na janela ao gráfico.	8
3.1	Mapa de memória visto pelo servidor: memórias de dados atrás da ponte HPS→FPGA e registradores de controle atrás da ponte leve. Os endereços são deslocamentos a partir da base de cada ponte.	11
3.2	O que há dentro da FPGA: quatro aceleradores, cada um com as suas memórias e os seus registradores de controle.	12
3.3	O bloco IIR: cascata de seções de segunda ordem (acima) e a seção em forma direta I (abaixo). Um só multiplicador-acumulador atende todas as seções, dois ciclos por seção.	14
3.4	O ADC da placa: o que existe (linha cheia) e o que falta para capturar sinais a uma taxa fixa (tracejado).	15
5.1	Os cabeçalhos de requisição e de resposta, em bytes.	18
6.1	Os dois formatos de ponto fixo, bit a bit.	19
7.1	<i>Overlap-add</i> : cada bloco de L amostras é convoluído na placa; as caudas de $M - 1$ amostras se somam.	20
7.2	FFT de $N = M \cdot 1024$ pontos pelo algoritmo de quatro passos: M FFTs de 1024 na placa, fatores de giro e DFTs de M pontos no PC.	21
7.3	IIR por blocos: cada bloco começa algumas amostras antes (<i>aquecimento</i>) e essa parte da saída é descartada.	21
8.1	Escolha da placa: o cliente sonda todas em paralelo e fica com a que responder primeiro. Uma placa ocupada só responde ao fim da operação em curso.	22
9.1	O que o <code>morphe-up.sh</code> faz, em ordem.	23

Capítulo 1

Introdução

1.1 O que é o Morphe

O Morphe é uma plataforma para executar operações de Processamento Digital de Sinais (PDS) em hardware e comparar o resultado com o que o computador calcula. O aluno prepara um sinal numa janela do aplicativo, escolhe a operação e recebe de volta duas respostas lado a lado: a que a FPGA de uma placa DE1-SoC calculou em ponto fixo e a que o próprio computador calculou em ponto flutuante. A diferença entre as duas é o objeto de estudo.

São cinco as operações executadas em hardware:

Convolução linear de dois sinais de até 1024 amostras cada, com saída de até 2047 amostras.

Filtro FIR a mesma convolução, com a resposta ao impulso vinda de um projetista de filtros por especificação.

Filtro IIR cascata de até 16 seções de segunda ordem (ordem até 32), também com projetista próprio.

FFT transformada de Fourier discreta de 1024 pontos.

IFFT a transformada inversa, no mesmo núcleo da FFT.

O hardware trabalha com blocos de tamanho fixo (1024 amostras), mas o aplicativo aceita sinais de qualquer tamanho: divide o sinal em blocos, manda cada um à placa e recompõe o resultado, com técnicas que dão a mesma resposta que uma operação única daria (capítulo 7).

1.2 Para que serve no ensino de PDS

Nas disciplinas de PDS as operações são estudadas com números reais, e o computador as calcula em ponto flutuante de 64 bits, que para quase todo fim didático é o mesmo que calcular com números reais. Um circuito digital dedicado não trabalha assim: usa ponto fixo, com um número pequeno de bits para a parte fracionária, e cada multiplicação precisa ser arredondada ou truncada.

O Morphe torna essa diferença visível e mensurável. Com ele o aluno pode:

- ver o erro de quantização de uma FFT em decibéis e observar que ele melhora 6 dB a cada bit de faixa aproveitada (capítulo 6);
- provocar uma saturação ou um estouro de acumulador e reconhecer o sintoma na saída;
- comparar um filtro IIR projetado em ponto flutuante com o mesmo filtro executado em ponto fixo, e entender por que o hardware usa seções de segunda ordem e não um polinômio de ordem alta;

- medir quanto tempo cada operação leva na placa e relacionar esse tempo com o número de operações do algoritmo.

1.3 Histórico

O Morphe nasceu como Trabalho de Conclusão de Curso de Carlos Valadão na UEFS, *Implementação de Ferramentas de Computação Numérica para o Processamento Digital de Sinais em Hardware Reconfigurável*. A arquitetura, o núcleo de convolução `conv1d`, a integração do núcleo de FFT, o servidor em C e o cliente gráfico em Python são dele, e o projeto é distribuído sob a licença GNU GPL versão 3.

A partir de setembro de 2026 a plataforma foi continuada num estágio curricular no LabPS/UEFS, por Antonio Nicassio Santos Lima, sob supervisão do Prof. Armando S. Sanca. O objetivo do estágio foi tornar a plataforma reproduzível a partir do repositório e utilizável por uma turma de alunos que não conhece os detalhes de baixo nível. A Tabela 1.1 resume o que mudou; o registro dia a dia está em `docs/DIARIO.md` e as correções feitas no projeto original, com os motivos, em `docs/RESSALVAS.md`.

Tabela 1.1: O que o estágio acrescentou ao projeto original.

	Projeto original	Hoje
Tamanho	128 amostras na convolução e no FIR	1024 amostras em todas as operações, e sinais maiores por blocos
Operações	convolução, FIR, FFT	as três, mais IFFT e IIR
Filtros	sem projetista por especificação	projetistas FIR e IIR por especificação
Precisão	entrada da FFT sem normalização	normalização da entrada: de 50 para 92 dB de SNR na FFT
Preparar a placa	passos manuais, conhecidos pelo autor	um comando, <code>morphe-up.sh</code>
Laboratório	uma estação, uma placa	duas placas, vários clientes ao mesmo tempo, instalação para a turma e Quartus em todos os PCs
Reprodutibilidade	o que funcionava estava só na estação	tudo sai do repositório, com manifesto de integridade

1.4 Visão geral

A Figura 1.1 é a figura de referência deste manual: todo o resto detalha uma das partes dela.

Cada **PC do laboratório** roda o aplicativo Morphe, escrito em Python. Ele não precisa de nenhum software da Intel: fala com as placas só pela rede.

Cada **placa DE1-SoC** tem, no mesmo chip, um processador ARM rodando Linux (o HPS, *Hard Processor System*) e uma FPGA. No processador roda o `morphe_server`, que recebe as requisições pela rede na porta 5000; na FPGA estão os aceleradores que fazem as contas. Os dois lados conversam por pontes internas ao chip (seção 3.2).

A **estação** é o computador ligado às placas pelo cabo USB-Blaster. É dela que se roda o `morphe-up.sh`, que programa a FPGA pelo JTAG, compila e reinicia o servidor na placa por SSH e registra a placa para os clientes. A estação também roda o aplicativo, como qualquer outro PC.

1.5 Como ler este manual

Quem só vai **usar** a plataforma precisa do capítulo 7 (o aplicativo) e do capítulo 9 (preparar as placas). Quem vai **estender** a plataforma — acrescentar uma operação, mudar um acelerador,

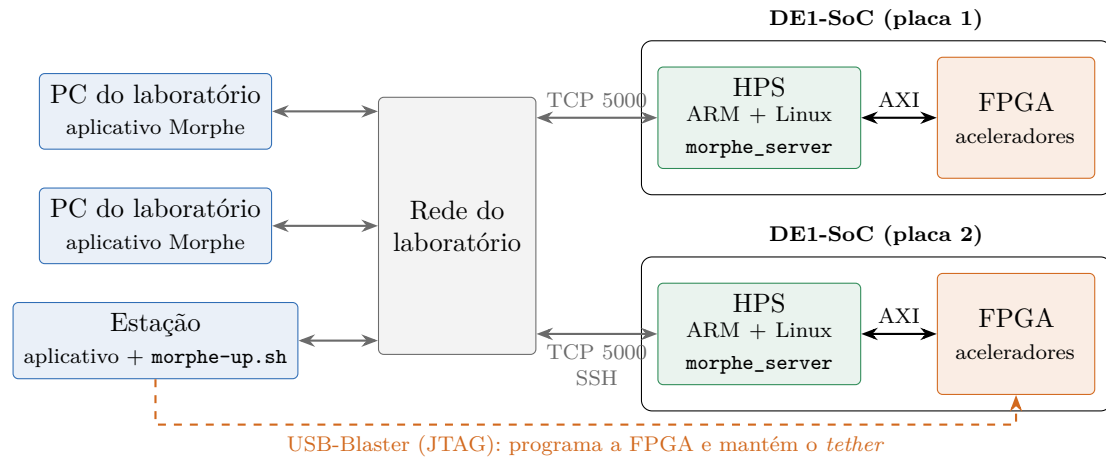


Figura 1.1: Visão geral: os computadores do laboratório falam com as placas pela rede (TCP, porta 5000); a estação programa a FPGA pelo USB-Blaster e sobe o servidor por SSH.

recompilar o hardware — deve ler os capítulos em ordem: a arquitetura (2), o hardware (3), o servidor (4), o protocolo (5) e a representação numérica (6) são o que se precisa saber antes de mexer em qualquer um deles.

Convenções usadas no texto:

- nomes de arquivos, comandos e sinais do hardware aparecem em **fonte monoespaçada**; caminhos são relativos à raiz do repositório;
- endereços e constantes do hardware aparecem em hexadecimal, com o prefixo `0x`;
- $Qm.n$ é um número em ponto fixo com sinal, m bits de parte inteira e n de parte fracionária (capítulo 6);
- todo número medido vem com a condição em que foi medido. Um número sem condição é de especificação, não de medida.

Capítulo 2

Arquitetura em três camadas

2.1 As camadas e o que cabe a cada uma

O Morphe se divide em três camadas, uma por máquina ou por tipo de circuito (Figura 2.1). Cada uma é escrita numa linguagem e tem uma responsabilidade que as outras não repetem.

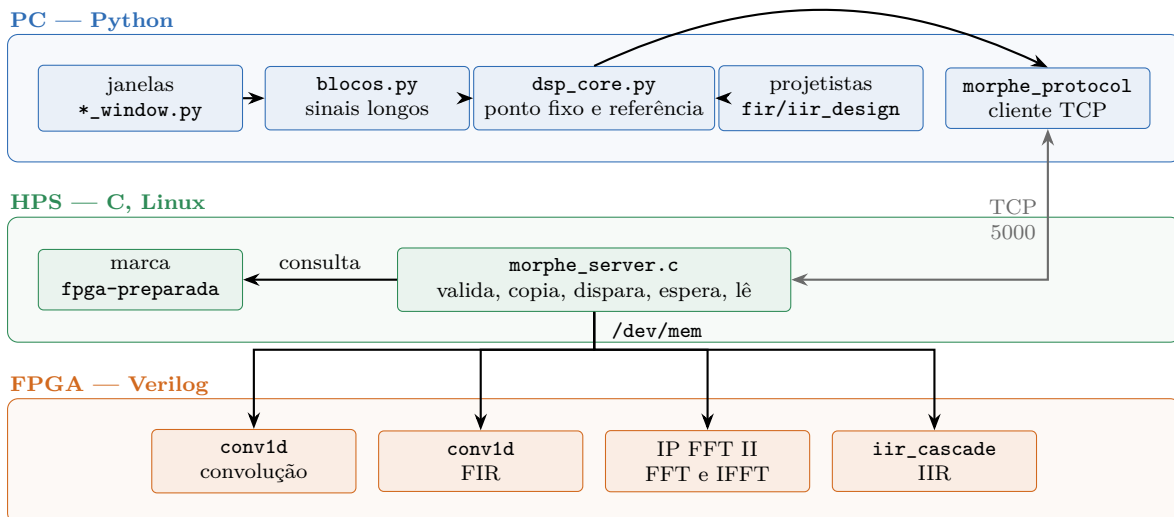


Figura 2.1: As três camadas e os módulos de cada uma.

Tabela 2.1: Responsabilidades de cada camada.

		Faz	Não faz
Cliente	Python, no PC	gera ou lê o sinal; projeta filtros; converte para ponto fixo; divide sinais longos em blocos; calcula a referência em ponto flutuante; compara e mostra	não sabe endereços de memória nem detalhes da FPGA
Servidor	C, no HPS da placa	valida a requisição; copia os dados para as memórias da FPGA; dispara o acelerador; espera; devolve o resultado	não converte formatos numéricos (com uma exceção na FFT, seção 3.3.3); não guarda nada entre requisições
Aceleradores	Verilog, na FPGA	fazem a conta, em ponto fixo, sobre blocos de tamanho fixo	não conhecem a rede nem o tamanho do sinal que o aluno pediu

Três princípios que atravessam as camadas

O servidor não guarda estado. Cada operação chega completa — os dados e todos os parâmetros — e sai completa. Não há sessão, reserva de placa nem memória da requisição anterior. É isso que permite que vários clientes usem a mesma placa intercalando operações, e que um cliente possa mudar de placa de uma operação para a outra sem nada a transferir.

O ponto fixo é decidido no cliente. O cliente converte o sinal para o formato do acelerador (Q15.16 ou Q15.8) antes de enviar, e converte o resultado de volta. O servidor copia inteiros de 32 bits sem interpretá-los, com uma única exceção: aplicar o expoente de bloco da FFT, que só ele conhece (seção 3.3.3). Fora isso, toda decisão numérica — normalizar a entrada, onde saturar, que fator aplicar à IFFT — mora num lugar só, em Python, onde é fácil de ler e de testar.

O hardware tem tamanho fixo; o software se adapta. Os aceleradores foram sintetizados para 1024 amostras. O servidor completa com zeros o que o cliente não mandou e devolve só as amostras que fazem sentido; o cliente, por sua vez, divide sinais maiores em blocos de 1024. Nenhum dos dois lados exige que o aluno pense em 1024.

Os números de configuração existem em dois lugares

Os limites do hardware — 1024 amostras, 16 seções de IIR, os bits fracionários de cada formato — estão escritos em `C/morphe_config.h`, para o servidor, e repetidos em `Python/morphe_config.py`, para o cliente. Os dois arquivos precisam mudar juntos. Os endereços das memórias e registradores, ao contrário, não são escritos à mão: vêm do `C/hps_0.h`, gerado a partir do projeto do hardware (seção 3.2).

2.2 O caminho de uma operação

A Figura 2.2 acompanha uma convolução do clique na janela até o gráfico. As outras operações seguem o mesmo roteiro, mudando só as memórias e o formato numérico.

1. **A janela entrega os sinais.** $x[n]$ e $h[n]$ chegam ao módulo de protocolo como vetores em ponto flutuante, do tamanho que o aluno escolheu.
2. **O cliente codifica.** Cada amostra vira um inteiro de 32 bits em Q15.16: $q = \text{round}(v \cdot 2^{16})$, saturado na faixa do formato. Se o sinal passa de 1024 amostras, a divisão em blocos acontece antes deste passo, e os passos seguintes se repetem por bloco.
3. **Uma conexão por operação.** O cliente abre uma conexão TCP com a placa, envia um cabeçalho de 20 bytes (operação, tipo dos dados, tamanhos) e os dados logo em seguida (capítulo 5).
4. **O servidor valida.** Confere a assinatura e a versão do protocolo, a operação, os tamanhos contra os limites do hardware e se a FPGA foi preparada pelo `morphe-up.sh` (capítulo 4). Qualquer falha vira uma resposta de erro com uma mensagem legível, e a FPGA não é tocada.
5. **O servidor escreve nas memórias.** Copia x e h para as memórias de entrada do acelerador, através da ponte HPS→FPGA, completando com zeros até 1024 amostras.
6. **O servidor dispara.** Escreve 1 no registrador `start` do acelerador, através da ponte leve. O acelerador detecta a subida desse sinal e começa.

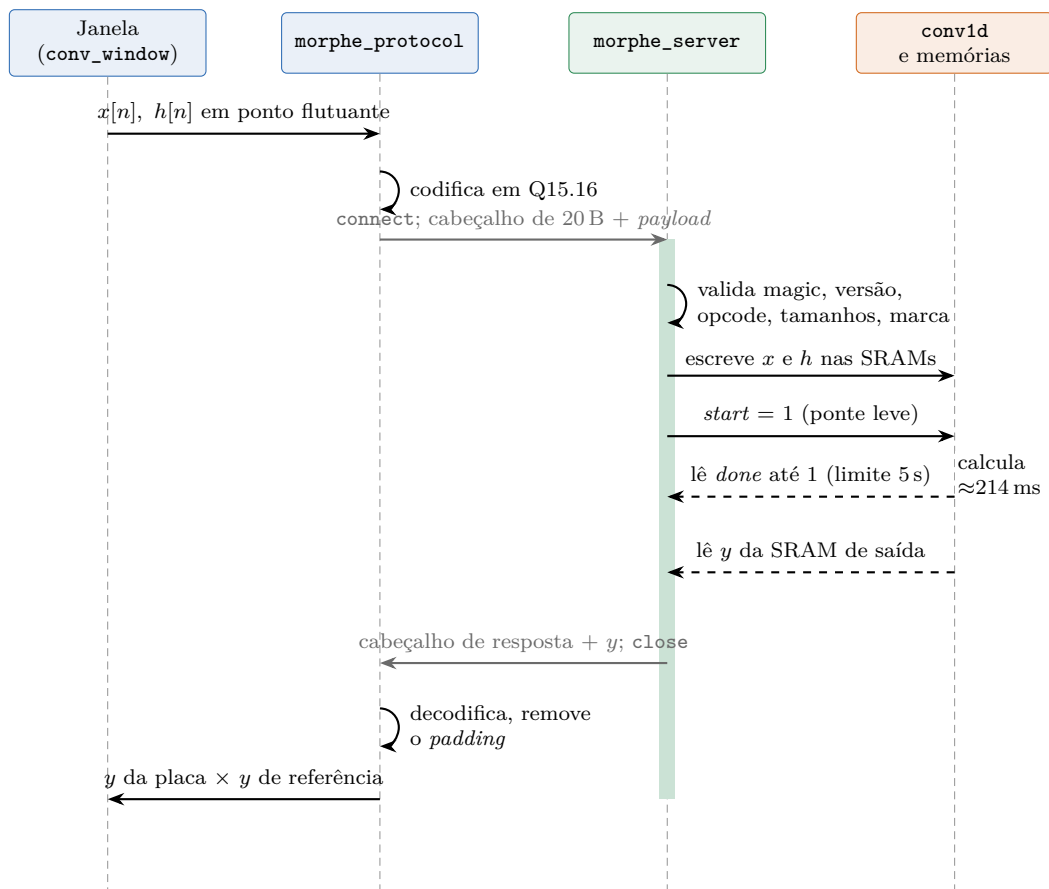


Figura 2.2: O caminho de uma operação, do clique na janela ao gráfico.

7. **O acelerador calcula** e, ao terminar, leva o sinal **done** a 1. A convolução de 1024 por 1024 amostras leva cerca de 214 ms; a FFT de 1024 pontos, cerca de 21 ms (tempos de ida e volta medidos pelo `morphe_ping` em 15/09/2026, com a placa sem outros clientes).
8. **O servidor espera.** Lê o **done** a cada 50 μ s. Se ele não subir em 5 s, a operação é abandonada com o erro de tempo esgotado — o sintoma típico de um bitstream que parou de funcionar (seção 3.3).
9. **O servidor lê e responde.** Copia as $n_x + n_h - 1$ amostras úteis da memória de saída, devolve o **start** a 0 e envia o cabeçalho de resposta e os dados. Fecha a conexão.
10. **O cliente decodifica e compara.** Converte de volta para ponto flutuante, descarta o que for enchimento de zeros e calcula a referência em ponto flutuante com o NumPy. A janela mostra as duas e o erro entre elas.

Dois fatos decorrem desse roteiro e aparecem na prática.

O tempo de uma operação **não depende do tamanho do sinal pedido**: o acelerador sempre processa os blocos completos de 1024 amostras, e os zeros de enchimento custam o mesmo que amostras de verdade. Pela construção do `conv1d`, uma convolução de 10 por 10 amostras custa o mesmo que uma de 1024 por 1024.

Como o servidor atende **uma conexão de cada vez**, duas pessoas na mesma placa não recebem erro: a segunda espera a primeira terminar, e o tempo de cada uma soma. O capítulo 8 trata disso.

2.3 Onde está cada camada no repositório

Camada	Onde
Cliente	Python/ (aplicativo e módulos); testes em Python/ferramentas/
Servidor	C/ (<code>morphe_server.c</code> e cabeçalhos)
Aceleradores	Quartus/ (RTL, projeto do Platform Designer, bitstream)
Operação	<code>morphe-up.sh</code> , <code>instala-quartus.sh</code> , <code>deploy.sh</code> , na raiz

A árvore completa está no apêndice A.

Capítulo 3

O hardware

3.1 A placa DE1-SoC

A DE1-SoC, da Terasic, é construída em torno de um Cyclone V SoC da Intel (5CSEMA5F31C6). O “SoC” do nome quer dizer que o chip reúne duas coisas bem diferentes:

HPS (*Hard Processor System*): um processador ARM Cortex-A9 de dois núcleos, com memória e periféricos próprios, que roda um Linux gravado no cartão SD da placa. No Morphe ele cuida da rede e roda o servidor.

FPGA lógica programável, configurada pelo bitstream que o Quartus gera. No Morphe ela contém os aceleradores e as memórias em que eles trabalham.

Os dois lados estão no mesmo chip e se comunicam por pontes internas, sem passar por nenhum fio da placa. É essa proximidade que permite ao servidor escrever mil amostras numa memória da FPGA como se escrevesse num vetor em C.

A lógica da FPGA roda com o relógio de 50 MHz da placa (`CLOCK_50`). A análise de tempo da última compilação (15/09/2026) mede uma frequência máxima de 60,99 MHz para esse domínio: há folga de cerca de 20 %. A Tabela 3.1 mostra quanto do chip o projeto ocupa.

Tabela 3.1: Recursos da FPGA ocupados pelo projeto (compilação de 15/09/2026).

Recurso	Usado	Disponível	Ocupação
Elementos lógicos (ALMs)	11 140	32 070	35 %
Blocos DSP (multiplicadores)	30	87	34 %
Blocos de memória M10K	165	397	42 %

O que sobra é bastante: cerca de 230 blocos M10K livres, pouco menos de 300 KB, o suficiente para uma memória de captura de dezenas de milhares de amostras quando a aquisição pelo ADC for implementada (seção 3.4).

O bitstream e a licença da FFT

O bitstream versionado é `Quartus/output_files/soc_system_time_limited.sof`, compilado com o Quartus Prime Lite 20.1. O núcleo de FFT é um IP da Intel que a edição Lite só permite usar em modo de avaliação: o bitstream funciona **enquanto o programador JTAG continuar conectado** à placa — o *tether* — e para depois de cerca de uma hora sem ele. Isso vale para o bitstream inteiro, não só para a FFT: sem o *tether*, a convolução também passa a esgotar o tempo. O `morphe-up.sh` mantém o *tether* vivo (capítulo 9).

3.2 Pontes e mapa de memória

O HPS enxerga a FPGA como faixas de endereços. O Morphe usa duas pontes (Figura 3.1):

Ponte HPS→FPGA (`h2f_axi_master`), a partir de `0xC000 0000`. Larga e rápida, é por onde passam os dados: as memórias de entrada e de saída de cada acelerador.

Ponte leve (*lightweight*), a partir de `0xFF20 0000`. Estreita, feita para registradores de controle: os PIOs de `start`, `done` e de parâmetros.

Ponte HPS→FPGA base 0xC000 0000, 32 bits		Ponte leve (LW) base 0xFF20 0000, PIOs	
0x10000	fir_yn	0x00	fir_error
0x12000	convld_yn	0x10	fir_done
0x14000	fir_hn	0x20	fir_start
0x15000	fir_xn	0x30	convld_done
0x16000	convld_hn	0x40	convld_start
0x17000	convld_xn	0x50	fft_bfp_exponent
0x18000	fft_yn_re	0x60	fft_wrapper_done
0x19000	fft_yn_imag	0x70	fft_inverse
0x1A000	fft_xn_re	0x80	fft_wrapper_start
0x1B000	fft_xn_imag	0x90	iir_start
0x1C000	iir_xn	0xA0	iir_done
0x1D000	iir_yn	0xB0	iir_error
0x1E000	iir_coef	0xC0	iir_nsecoes

convolução e FIR
 FFT e IFFT
 IIR

Figura 3.1: Mapa de memória visto pelo servidor: memórias de dados atrás da ponte HPS→FPGA e registradores de controle atrás da ponte leve. Os endereços são deslocamentos a partir da base de cada ponte.

As memórias têm duas portas. Cada uma das treze memórias de dados é uma RAM interna da FPGA de porta dupla: uma porta está ligada à ponte, para o HPS, e a outra ao acelerador. O servidor e o acelerador nunca disputam a mesma porta; o que os separa no tempo é o protocolo de `start` e `done` (seção 3.3). As memórias de entrada guardam 1024 palavras de 32 bits (4 KB); as de saída da convolução e do FIR, 2047 (8 KB); a de coeficientes do IIR, 5 palavras por seção para até 16 seções.

Como o servidor chega lá. O servidor abre `/dev/mem` e mapeia as duas faixas no próprio espaço de endereços com `mmap`; a partir daí, escrever `g_conv_xn[i]` é escrever na memória da FPGA. Por isso ele precisa rodar como `root`. O tamanho da faixa mapeada é calculado em tempo de compilação a partir do `hps_0.h`, e verificações estáticas no código recusam compilar se alguma memória for menor do que os limites de `morphe_config.h` exigem.

O `hps_0.h` é gerado, nunca editado. Os endereços da Figura 3.1 são decididos no Platform Designer, a ferramenta do Quartus que monta o sistema, e ficam registrados no arquivo `soc_system.sopcinfo`. O `Quartus/gen_hps_header.py` lê esse arquivo e escreve o `C/hps_0.h`; com `--check`, apenas confere se o cabeçalho versionado ainda corresponde ao projeto. Um

servidor compilado com endereços velhos escreve nos lugares errados sem dar erro nenhum; a conferência existe para que isso não aconteça.

Cuidado com o mapa do “DE1-SoC Computer”. Os exemplos da Intel para a placa usam 0xC800 0000 como base da memória da FPGA. No Morphe a base é 0xC000 0000. Código copiado desses exemplos lê e escreve fora das memórias do projeto (docs/RESSALVAS.md, item 11).

3.3 Os aceleradores

A Figura 3.2 mostra o que há dentro da FPGA: quatro aceleradores, cada um com as suas memórias e os seus registradores de controle, todos ligados ao HPS pela interconexão que o Platform Designer gera.

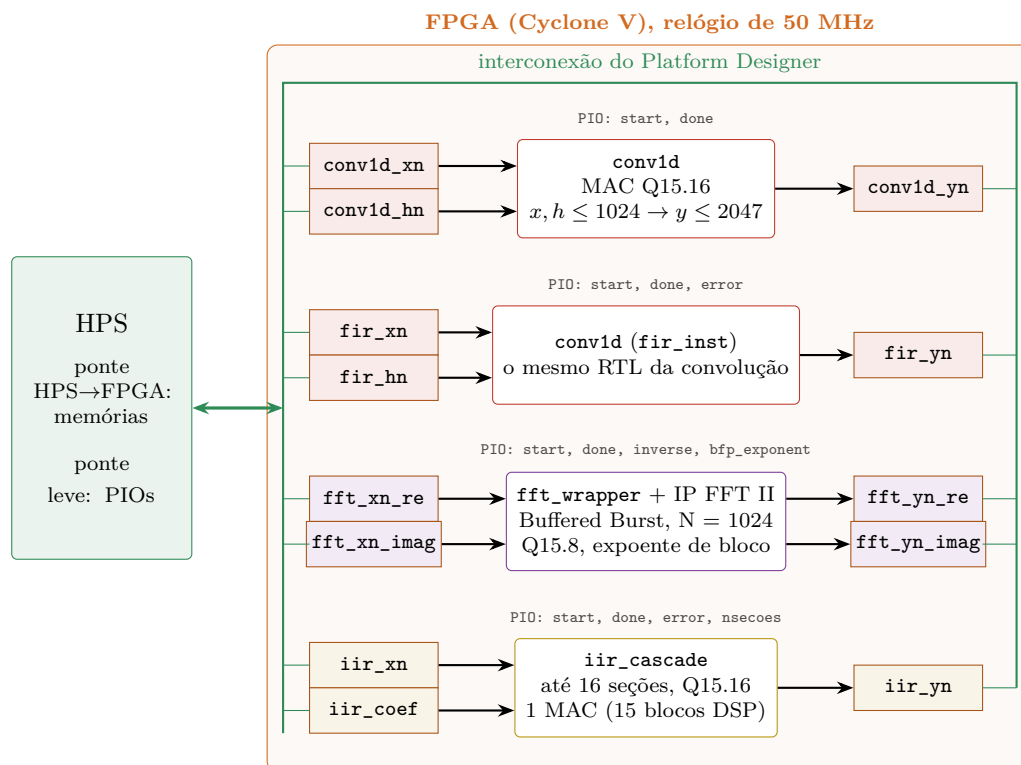


Figura 3.2: O que há dentro da FPGA: quatro aceleradores, cada um com as suas memórias e os seus registradores de controle.

3.3.1 O protocolo comum

Todos os aceleradores obedecem ao mesmo combinado com o servidor:

1. com **start** em 0, o servidor escreve as entradas nas memórias;
2. o servidor leva **start** a 1; o acelerador detecta a *subida* do sinal, zera o **done** e começa;
3. o acelerador lê as entradas, calcula, escreve as saídas e leva **done** a 1;
4. o servidor, que lê **done** a cada 50 μ s, lê as saídas e devolve **start** a 0.

Como o acelerador reage à subida e não ao nível, um **start** esquecido em 1 não dispara uma segunda execução. E como o servidor zera todos os **start** na primeira requisição depois de a

FPGA ser preparada, um valor que tenha sobrado de antes do bitstream do Morphe também não dispara nada.

Os aceleradores acessam as memórias por pequenos controladores que gastam alguns ciclos por acesso. É um custo deliberado de simplicidade: numa convolução, o tempo é dominado por esses acessos, não pela multiplicação. Os controladores estão em `memory_read_controller.v` e em `memory_write_controller.v`.

O tempo como sinal de saúde. Uma convolução de 1024 por 1024 amostras executa cerca de 8,4 milhões de ciclos, uns 168 ms a 50 MHz; de ponta a ponta, com a rede, o cliente mede 214 ms. Uma resposta muito mais rápida que isso é sinal de defeito, não de sorte: um bitstream mal compilado já foi visto levantar o `done` em 3,3 ms sem ter calculado nada (`docs/RESSALVAS.md`, item 13).

3.3.2 Convolução e FIR: o `conv1d`

O `conv1d.v` calcula a convolução linear

$$y[n] = \sum_k x[k] h[n - k], \quad n = 0, 1, \dots, 2046,$$

com os limites da soma ajustados a cada n para não ler fora dos vetores. É uma máquina de estados com um único multiplicador-acumulador: para cada n , lê $x[k]$ e $h[n - k]$, multiplica, acumula e, ao fim da soma, escreve $y[n]$.

A aritmética é em Q15.16. O produto de dois números de 32 bits tem 64 bits; o `conv1d` guarda os bits 47 a 16, o que realinha o ponto fracionário *truncando* os 16 bits de baixo. O acumulador tem 32 bits e *não satura*: quando a soma passa de ± 32768 ela dá a volta e aparece com o sinal trocado. Duas consequências práticas:

- o erro de truncamento se acumula com o número de termos: numa convolução de 1024 por 1024 ele chega a centenas de LSB. É esperado;
- com sinais de amplitude A e N termos, a saída estoura quando $NA^2 \geq 32768$ — com $N = 1024$, a partir de $A \approx 5,7$. Uma saída negativa para sinais positivos é esse estouro, não um defeito.

Trocar o truncamento por arredondamento reduziria o erro máximo medido de 521 para 25 LSB. A troca foi avaliada e **deliberadamente não feita** (14/09/2026): o comportamento numérico do `conv1d` é o que o TCC original descreve, e ele fica registrado em `docs/RESSALVAS.md` como escolha, não como defeito esquecido.

O FIR é o mesmo circuito. O filtro FIR é uma segunda instância do `conv1d` (`fir_inst`), com memórias próprias. Filtrar um sinal por um FIR é convoluí-lo com a resposta ao impulso do filtro, e é exatamente isso que acontece; a diferença entre as duas operações está no aplicativo, que oferece para o FIR um projetista de filtros. O projeto ainda contém um IP de FIR da Intel (`fir_ii`) e o seu adaptador, `fir_wrapper.v`, mas a instância está comentada no `ghrd_top.v`. O registrador `fir_error` sobrou desse adaptador e o `conv1d` não o aciona.

3.3.3 FFT e IFFT: o núcleo da Intel

A transformada usa o IP *FFT II* da Intel, configurado para 1024 pontos em modo *Buffered Burst*, com entrada de 24 bits e fatores de giro de 18 bits. O `fft_wrapper.v` faz a ponte entre as memórias e o IP: lê as partes real e imaginária da entrada, entrega-as ao IP amostra por amostra respeitando o *handshake* do barramento de fluxo, recebe o resultado e o escreve nas

memórias de saída. O registrador `fft_inverse` escolhe entre a transformada direta e a inversa; o resto do caminho é o mesmo.

Duas particularidades desse núcleo aparecem no resultado:

Expoente de bloco. A cada estágio, o IP detecta se os valores cresceram e, se preciso, desloca as 1024 amostras *juntas*, acumulando um expoente único para o bloco (*block floating point*). O adaptador captura esse expoente no registrador `fft_bfp_exponent`, e o servidor o aplica, junto com o fator 2^{-8} do formato Q15.8, antes de devolver o resultado em ponto flutuante. É a única conversão numérica feita no servidor.

IFFT sem o fator $1/N$. O núcleo entrega a inversa não normalizada, N vezes a IDFT matemática. O cliente aplica o $1/1024$.

O capítulo 6 explica o que o expoente de bloco e os 24 bits de entrada significam para a precisão, e por que ela para em torno de 92 dB.

3.3.4 IIR: a cascata de seções de segunda ordem

O filtro IIR é a única operação em que a saída depende das saídas anteriores, e isso muda o que importa no projeto do circuito. Ele é implementado como uma cascata de até 16 seções de segunda ordem (*biquads*), cada uma em forma direta I (Figura 3.3):

$$y_s[n] = b_0 x_s[n] + b_1 x_s[n-1] + b_2 x_s[n-2] - a_1 y_s[n-1] - a_2 y_s[n-2],$$

com a saída de cada seção servindo de entrada para a seguinte. Os coeficientes vêm da memória `iir_coef`, cinco por seção, na ordem b_0, b_1, b_2, a_1, a_2 ($a_0 = 1$ fica implícito); o número de seções, do registrador `iir_nsecoes`.

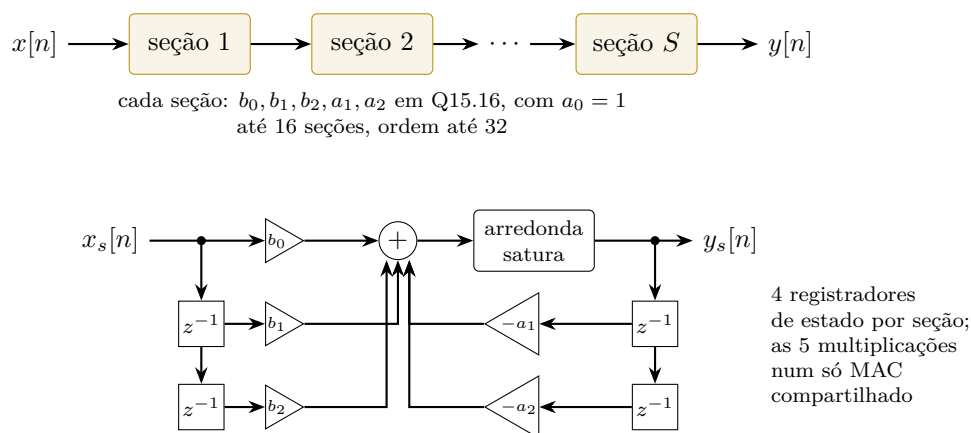


Figura 3.3: O bloco IIR: cascata de seções de segunda ordem (acima) e a seção em forma direta I (abaixo). Um só multiplicador-acumulador atende todas as seções, dois ciclos por seção.

Cada escolha desse projeto foi medida antes de ser feita:

Seções de segunda ordem, e não um polinômio de ordem N . A posição dos polos de um polinômio de ordem alta é extremamente sensível aos coeficientes. Um Butterworth de ordem 22, estável por construção, volta do próprio MATLAB com um polo de módulo 1,278 — instável — porque achar as raízes daquele polinômio já é mal condicionado em ponto flutuante de 64 bits. A mesma especificação em seções de segunda ordem tem o maior polo em 0,988.

Forma direta I. Tem um acumulador único e largo e arredonda uma vez só, na saída da seção. A forma direta II transposta, melhor em ponto flutuante, guarda estados já arredondados, e numa realimentação esse erro circula.

Arredondar, e não truncar. Truncar introduz um viés de meio LSB negativo por amostra. Num filtro realimentado esse viés não some: vira um deslocamento DC de $-0,5/(1+a_1+a_2)$ LSB na saída. No FIR não importa, e por isso o `conv1d` pôde manter o truncamento.

Q15.16, o mesmo formato do `conv1d`. É suficiente: o raio dos polos se desloca no máximo cerca de 10^{-6} ao quantizar, mesmo com polos em $|p| = 0,9984$.

Uma amostra atravessa todas as seções antes da próxima. Assim bastam quatro registradores de estado por seção e uma única passagem pela memória. O resultado é idêntico, bit a bit, ao de processar seção por seção, porque cada seção só vê a saída já arredondada da anterior.

Um só multiplicador-acumulador, compartilhado. As cinco multiplicações de 32 por 32 bits de uma seção são feitas juntas, e o mesmo circuito atende as 16 seções em sequência. O custo em blocos DSP (15, porque cada multiplicação de 32 bits ocupa três multiplicadores de 27 bits) não depende do número de seções. Cada seção leva dois ciclos: um para selecionar os operandos, outro para calcular. Com um ciclo só, o circuito não fechava a 50 MHz (49,11 MHz medidos em 14/09/2026); com dois, fecha a 60,99 MHz.

Se alguma amostra satura, o registrador `iir_error` sobe; o resultado é devolvido mesmo assim, saturado, e o cliente avisa o aluno. O estado interno é zerado a cada execução, para que o mesmo sinal dê sempre a mesma saída.

Verificado bit a bit. O RTL do IIR tem um modelo de referência em Python que usa inteiros de precisão arbitrária (o acumulador de uma seção chega a 2^{65} , além do que um `float64` representa exatamente). O testbench em Icarus Verilog (`Quartus/roda_tb_iir.sh`) e a placa (`testa_iir_hw.py`) são comparados com esse modelo e precisam coincidir em todas as amostras: num filtro realimentado, divergir um LSB numa amostra é divergir para sempre dali em diante, então “parecido” não serve de critério.

3.4 Periféricos ainda não usados: ADC e codec

A placa tem dois conversores que o Morphe ainda não usa. Esta seção descreve o estado de hoje e o que falta; a Figura 3.4 resume as duas coisas.

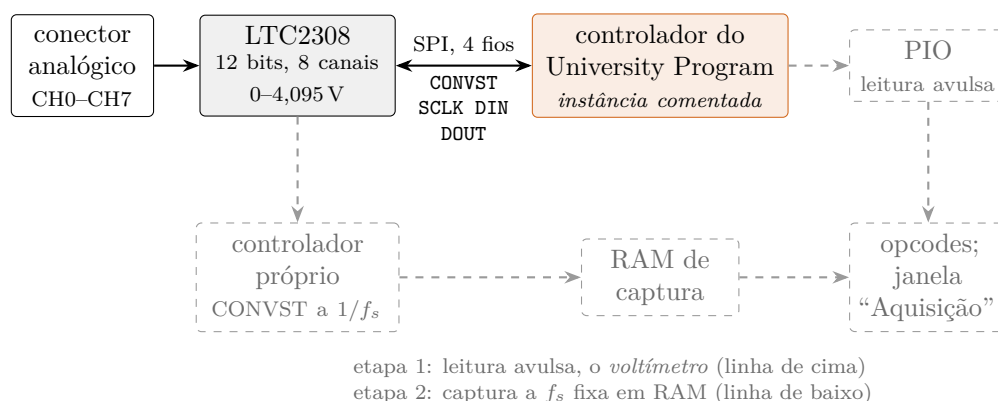


Figura 3.4: O ADC da placa: o que existe (linha cheia) e o que falta para capturar sinais a uma taxa fixa (tracejado).

O ADC. O conversor analógico-digital da DE1-SoC é o LTC2308: 12 bits, 8 canais, por aproximações sucessivas, ligado à FPGA por uma interface serial de quatro fios (`ADC_CONVST`, `ADC_SCLK`, `ADC_DIN`, `ADC_DOUT`). As entradas analógicas chegam por um conector próprio da placa. O projeto contém o controlador do *University Program* da Intel para esse conversor, configurado para entrada simples e unipolar: com a referência interna, a faixa é de 0 a 4,095 V, 1 mV por passo. **Tensão negativa não é lida e pode danificar a entrada;** um sinal alternado precisa de um deslocamento DC antes de chegar ao conector.

Hoje a instância desse controlador está comentada no `ghrd_top.v`: o ADC não faz parte do bitstream. E mesmo ativo, ele não serviria para amostrar um sinal: converte sem parar, sozinho, e entrega apenas o último valor lido de cada canal, sem controle do instante da conversão. Uma incerteza de alguns microssegundos no instante da amostra limita a medida de um tom de 5 kHz a cerca de 25 dB. Capturar a uma taxa de amostragem fixa exige um controlador próprio, que dispare a conversão a cada $1/f_s$ e guarde as amostras numa memória da FPGA. O plano, em duas etapas — uma leitura avulsa, como um voltímetro, e depois a captura temporizada — está em `docs/ADC.md`.

O codec de áudio. A placa tem também um codec de áudio (o WM8731, segundo a documentação da DE1-SoC), com entrada e saída de linha, ligado à FPGA por uma interface de áudio serial e configurado por I²C. Os pinos estão declarados no `ghrd_top.v` e mais nada: nenhum sinal do codec é usado pelo projeto. Seria o caminho natural para uma saída de áudio — ouvir o sinal filtrado — e fica como trabalho futuro.

Capítulo 4

O servidor na placa

Conteúdo previsto. O `morphe_server.c`: mapeamento de `/dev/mem`, laço `accept-atende-fecha`, um cliente por vez, fila do sistema operacional (`listen(4)`). Validação do cabeçalho, tempo-limite de 5 s por operação. A marca `/var/run/morphe-fpga-preparada` e por que o servidor recusa operar sem ela (o travamento do barramento com o bitstream de fábrica, 21/09). Compilação na própria placa e autostart. Tabela dos códigos de status.

Capítulo 5

O protocolo Morphe-TCP

Conteúdo previsto. Uma conexão por operação. Cabeçalhos de 20 bytes em big-endian (Figura 5.1), tabela de opcodes (*CONV*, *FFT*, *PING*, *FIR*, *IFFT*, *IIR*) com o formato do payload de cada um, tabela de status. Como acrescentar um opcode: os quatro lugares que mudam juntos (`morphe_protocol.h`, `morphe_server.c`, `morphe_protocol.py`, `morphe_config.*`).

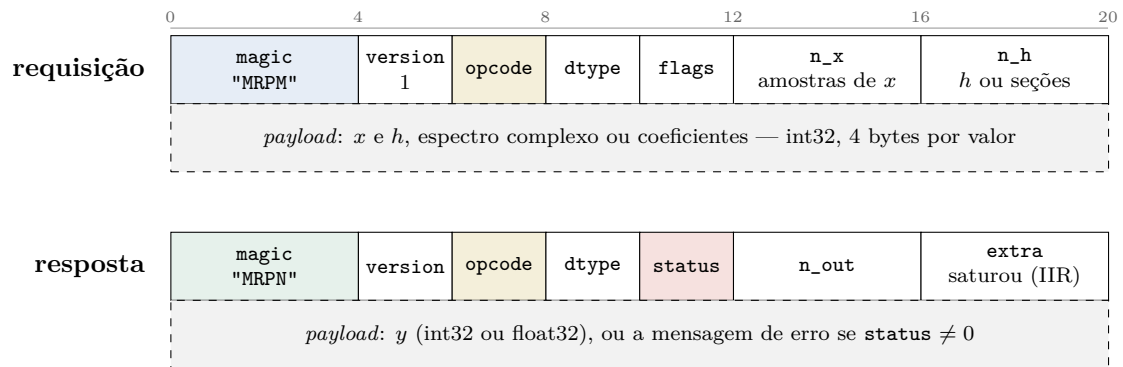


Figura 5.1: Os cabeçalhos de requisição e de resposta, em bytes.

Capítulo 6

Representação numérica

Conteúdo previsto. Os dois formatos de ponto fixo (Figura 6.1): $Q15.8$ na FFT e na IFFT, $Q15.16$ na convolução, no FIR e no IIR. A regra dos 6,02 dB por bit, a normalização da entrada e o ponto fixo em bloco do IP da FFT; por que o teto é ~ 92 dB. A IFFT sem o fator $1/N$. Tabela dos erros medidos e regras práticas. Resume docs/PRECISAO-NUMERICA.md.

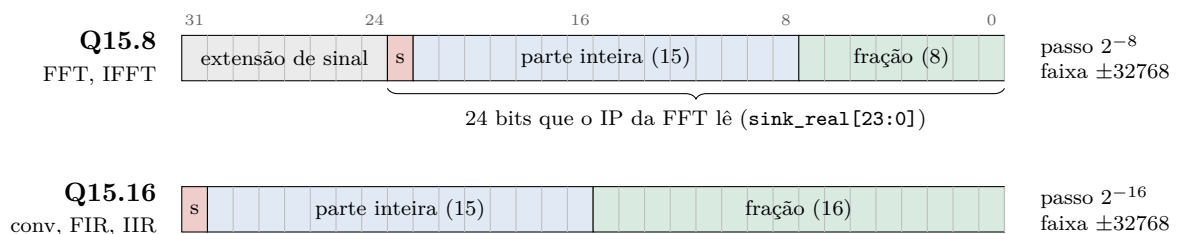


Figura 6.1: Os dois formatos de ponto fixo, bit a bit.

Capítulo 7

O aplicativo cliente

7.1 Janelas e módulos

Conteúdo previsto. O `morphe_app.py` e as janelas (gerador de sinais, convolução, FFT, IFFT, FIR com projetista, IIR com projetista, comparador), com uma captura de tela de cada — as únicas figuras que não são desenhadas em $\text{L}^{\text{A}}\text{T}_{\text{E}}\text{X}$. De onde vem o sinal: gerador, digitado ou arquivo (`.csv`, `.txt`, `.npy`, `.wav`). Os módulos por trás das janelas (`dsp_core`, `blocos`, `fir_design`, `iir_design`, `morphe_export`). O pacote `.mrph` e o comparador.

7.2 Sinais maiores que o hardware: processamento em blocos

Conteúdo previsto. O hardware processa 1024 amostras por vez; o cliente divide e recompõe. Três técnicas, uma por tipo de operação, cada uma com a sua figura e o seu argumento de exatidão: overlap-add para convolução e FIR; algoritmo de quatro passos para FFT e IFFT (a mesma DFT, não uma aproximação); blocos com aquecimento para o IIR (bit a bit igual ao modelo nos mesmos blocos).

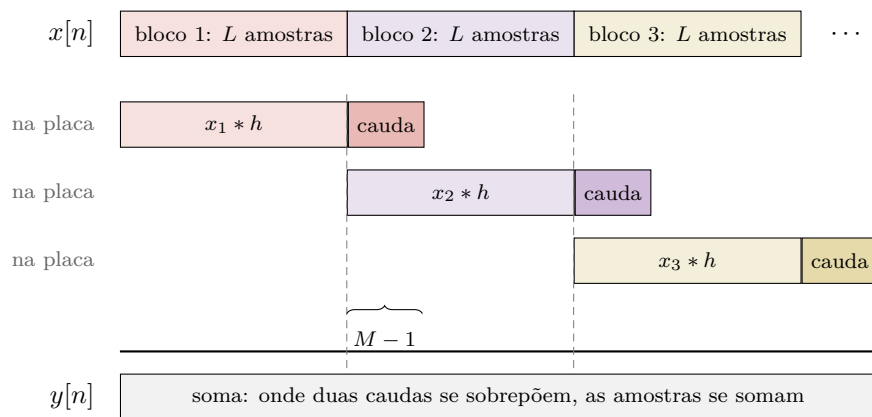
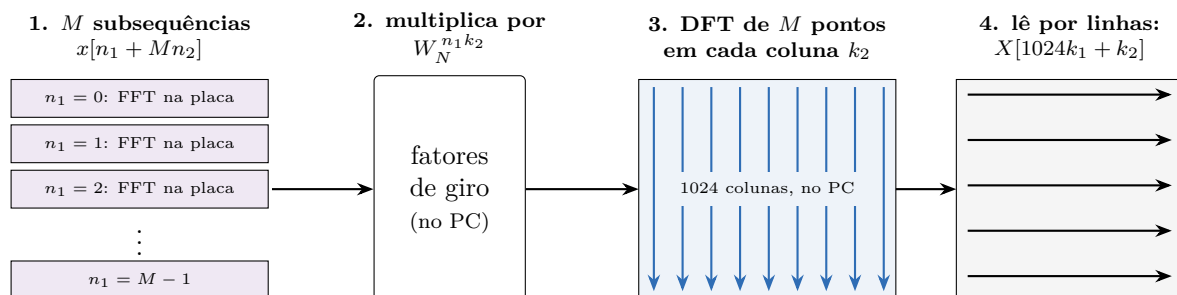


Figura 7.1: *Overlap-add*: cada bloco de L amostras é convoluido na placa; as caudas de $M - 1$ amostras se somam.



Não é aproximação: é a DFT de N pontos com a conta dividida (Cooley–Tukey com $N_1 = M$, $N_2 = 1024$). A IFFT longa usa a mesma decomposição, com os fatores conjugados e $1/N = 1/1024 \cdot 1/M$.

Figura 7.2: FFT de $N = M \cdot 1024$ pontos pelo algoritmo de quatro passos: M FFTs de 1024 na placa, fatores de giro e DFTs de M pontos no PC.

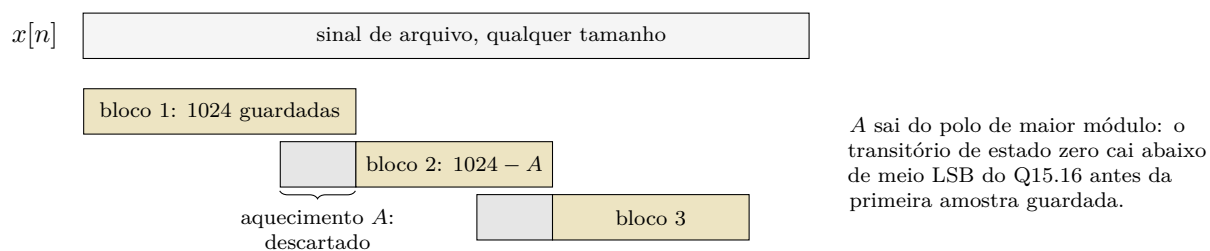


Figura 7.3: IIR por blocos: cada bloco começa algumas amostras antes (*aquecimento*) e essa parte da saída é descartada.

Capítulo 8

Várias placas, vários alunos

Conteúdo previsto. *Descoberta (lista em `.morphe-estado/placas`, varredura como último recurso), escolha pela sonda mais rápida (Figura 8.1), a fila do servidor e o que ela faz com a latência. Tabela das medições de 22/09 (6 clientes, 90/90 respostas conferidas). O que não existe: alocação de placas, concorrência no servidor, descoberta sem varredura. Resume docs/GERENCIAMENTO-PLACAS.md.*

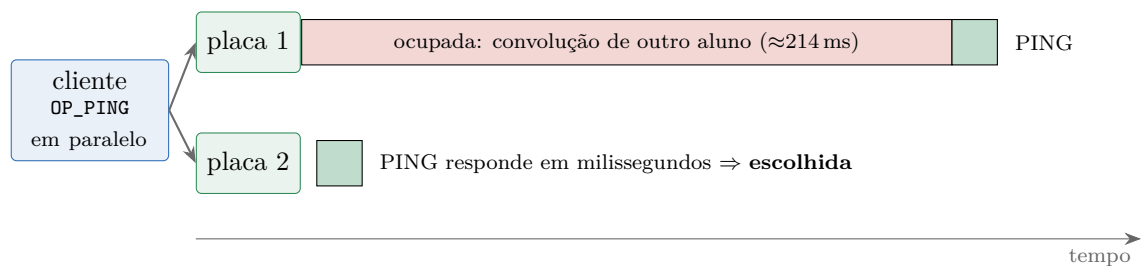


Figura 8.1: Escolha da placa: o cliente sonda todas em paralelo e fica com a que responder primeiro. Uma placa ocupada só responde ao fim da operação em curso.

Capítulo 9

Preparação e operação do laboratório

Conteúdo previsto. O `morphe-up.sh` passo a passo (Figura 9.1) e as suas opções (`--board`, `--skip-fpga`, `--down`, `--setup-ssh`); o tether da licença, um por cabo e por conta. A instalação de turma em `/opt/morphe` (conta do coordenador e conta dos alunos). O `instala-quartus.sh` e o Quartus 20.1 em todos os PCs. Incorporar uma placa nova (`docs/NOVA-PLACA.md`). Tabela sintoma → causa → o que fazer.

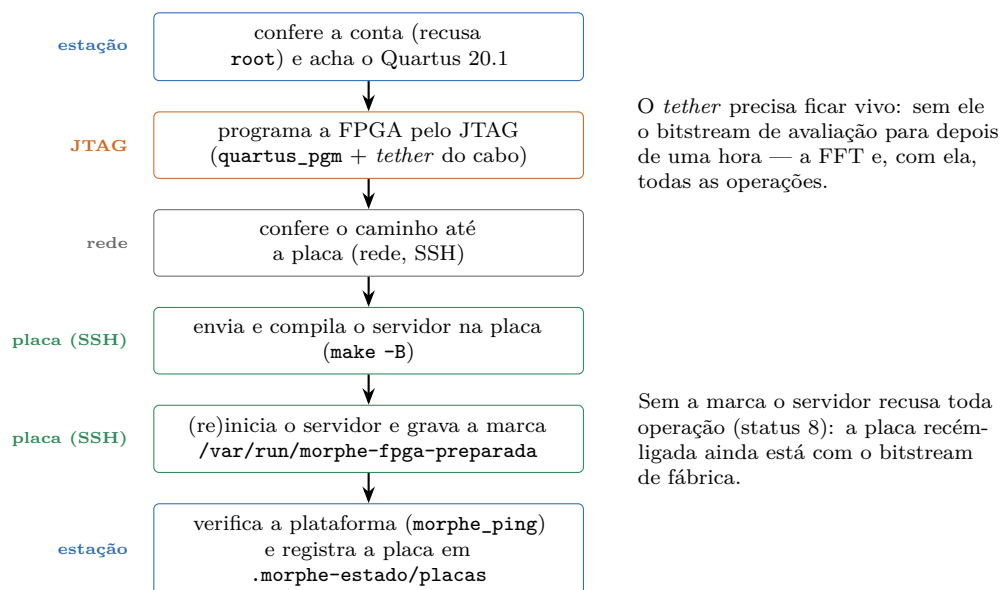


Figura 9.1: O que o `morphe-up.sh` faz, em ordem.

Capítulo 10

Desenvolvimento e verificação

Conteúdo previsto. *Para quem vai estender o sistema. Recompilar o bitstream (Platform Designer, Quartus, relatório de tempo; resume docs/COMPILAR-IIR.md). Testbenches em Icarus Verilog com vetores gerados em Python. Os testes contra a placa (testa_iir_hw, testa_blocos, testa_concorrencia, testa_ifft_roundtrip, testa_bit_inverse). A PROVENIENCIA.sha256: como provar que o que está na placa saiu do repositório. Fluxo de trabalho com o git (push do Windows, pull na estação).*

Capítulo 11

Limitações e trabalho futuro

Conteúdo previsto. *O que o sistema não faz hoje, cada item com o motivo: alocação de placas; aquisição pelo ADC (as etapas planejadas); saída de áudio pelo codec; licença de avaliação da FFT (o tether); tamanho fixo de 1024 no hardware.*

Apêndice A

Estrutura do repositório

Conteúdo previsto. A árvore abaixo, com uma linha por arquivo importante. A regra de que os `iir_*.v` não saem de Quartus/.

```
morphe/
├── README.md ..... ponto de entrada
├── morphe-up.sh ..... prepara a placa: FPGA, servidor, marca
├── instala-quartus.sh ..... Quartus 20.1 para todas as contas de um PC
├── deploy.sh, gera_proveniencia.sh, PROVENIENCIA.sha256
├── Python/ ..... o aplicativo
│   ├── morphe_app.py ..... janela principal
│   ├── *_window.py ..... uma janela por operação
│   ├── morphe_protocol.py ..... cliente TCP
│   ├── dsp_core.py, blocos.py ..... ponto fixo, referência, sinais longos
│   ├── fir_design.py, iir_design.py ..... projetistas de filtros
│   └── ferramentas/ ..... testes contra a placa, figuras, vetores
├── C/ ..... o servidor da placa
│   ├── morphe_server.c, morphe_protocol.h, morphe_config.h
│   ├── hps_0.h ..... gerado do Platform Designer
│   └── autostart/
├── Quartus/ ..... o hardware
│   ├── ghrd_top.v ..... topo: instancia os aceleradores
│   ├── conv1d.v, fft_wrapper.v, iir_*.v ..... os aceleradores
│   ├── soc_system.qsys, soc_system.qsf
│   ├── tb_iir_*.v, roda_tb_iir.sh ..... testbenches
│   └── output_files/ ..... o bitstream
├── exemplos/ ..... pacotes .mrph e sinais de exemplo
├── docs/ ..... documentação técnica e diário
└── legado/ ..... do projeto original, fora do produto
```

Apêndice B

Referência rápida

Conteúdo previsto. *Os comandos do dia a dia numa página: preparar a placa, abrir o aplicativo, conferir um clone, rodar os testes, desligar a plataforma.*